TEXAS A&M UNIVERSITY
ROBOTICS TEAM & LEADERSHIP EXPERIENCE

Week 7: Sensors, Filtering, and State Estimation

TURTLE Incubator - SO-ARM Software Stack

Agenda

Sensor Noise

- Where noise comes from on the SO-ARM and why it matters for control

Signal Processing Fundamentals

- Sampling, frequency concepts, and why filters work

Filters

- Low-pass, high-pass, band-pass, band-stop - what each one does

State Estimation

- Complementary filters, Kalman filters, and extended Kalman filters

Error Propagation

- How uncertainty compounds through a system

SO-ARM Application

- Filter encoder measurements and compare raw vs filtered

Demonstration + Hands-On + Deliverable

- Implement filtering on live SO-ARM joint data

What You Are Building Today

By end of session your script will:

- 1 Read raw encoder position from the SO-ARM at 50 Hz
- 2 Apply a low-pass IIR filter to smooth the signal
- 3 Apply a Kalman filter as a second alternative
- 4 Log both raw and filtered signals to CSV simultaneously
- 5 Plot raw vs low-pass vs Kalman on the same chart
- 6 Quantify the improvement: compare RMS noise before and after filtering

This is the exact pipeline running in every real robotics system before sensor data enters a controller.

The Week 6 PID controller you built will work better the moment you feed it filtered position data.

Sensor Noise

Sources of Sensor Noise in the SO-ARM

Three categories of noise hit the SO-ARM:

Encoder noise

- Quantization from discrete ticks
- Gear backlash causes position jitter

Electrical noise (EMI)

- Motor switching causes voltage spikes
- USB power supply fluctuation

Mechanical noise

- Vibration from other joints
- Link flex under load

All three corrupt the position reading the PID controller uses.



Measurement Error and Why It Matters

Noise is not just cosmetic - it directly damages control performance:

PID D term amplification

- Kd * d(error)/dt computes the derivative of noise if position is noisy
- High Kd + noisy sensor = motor chatter and heat, even at rest

State estimation errors

- FK and IK (Week 4) assume exact joint angles - noise makes position estimates wrong
- A 3-degree noise spike translates to centimeter-scale end-effector error

Compounding errors

- Noise in joint 1 propagates through the full transform chain to the end-effector
- For 6 joints, small errors multiply - state estimation errors grow with arm reach

The fix: filter position before it enters any other algorithm.

Signal Processing Fundamentals

Low-Pass Filtering - Concept

A low-pass filter lets slow changes through and blocks fast noise.

Frequency domain view:

- Useful joint motion is slow (0-5 Hz)
- Encoder noise is fast (20-200 Hz)
- Set cutoff between them - signal passes, noise blocked

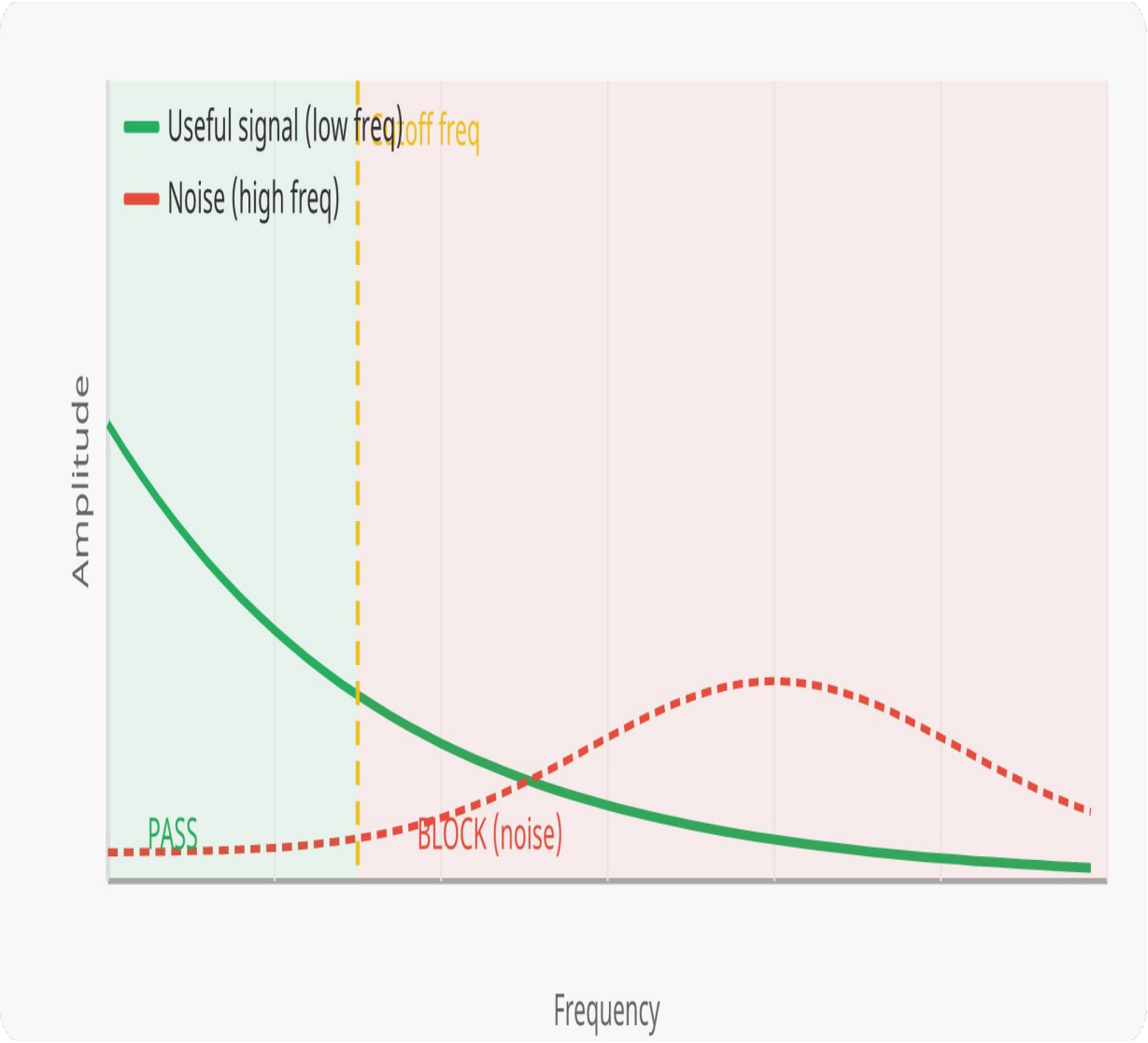
Green dashed = true signal

Red = raw noisy reading

Yellow = cutoff frequency

Everything to the left of the cutoff: PASS

Everything to the right of the cutoff: BLOCK



Sampling and the Nyquist Theorem

You can only detect frequencies up to half your sampling rate (Nyquist limit):

SO-ARM runs at 50 Hz control loop

- Maximum detectable frequency: 25 Hz
- Any noise above 25 Hz aliases (folds back) and appears as fake low-frequency content

Practical implication:

- If you run your loop at 10 Hz you will completely miss 15 Hz motor buzz
- Faster loops give you better visibility into what is actually happening

For the SO-ARM this session:

- Run at 50 Hz (dt=0.02s) - gives you up to 25 Hz frequency resolution
- Most encoder noise is above 10 Hz so low-pass at 5 Hz cutoff works well

Rule: sample at least 10x faster than the fastest motion you care about.

Filters

Filter Types

Four fundamental filter types:

Low-Pass
- Passes low freq, blocks high freq
- Use for: smoothing joint position

High-Pass
- Passes high freq, blocks low freq
- Use for: detecting impacts or collisions

Band-Pass
- Passes only a specific range
- Use for: detecting resonance at a known frequency

Band-Stop (Notch)
- Blocks only a specific frequency
- Use for: removing 60 Hz power hum

Filter Types - What Each One Passes and Blocks

Filter	Passes	Blocks	SO-ARM Use
Low-Pass (most common)	Low-frequency signal Smooth motion	High-freq noise and EMI Motor switching buzz	Joint position filtering
High-Pass	Fast changes and spikes High-freq events	Slow drift and DC bias Constant offsets	Collision detection
Band-Pass	Specific frequency range only	Everything outside the target band	Resonance detection
Band-Stop (notch filter)	Everything EXCEPT one specific frequency	A specific frequency (like 60 Hz power hum)	Remove 60Hz hum

For the SO-ARM this week: we implement Low-Pass and compare it to the Kalman filter.

The key parameter for low-pass is alpha (smoothing factor): 0 = frozen, 1 = no filtering.

Low-Pass IIR Filter - How It Works

IIR (Infinite Impulse Response) is the simplest practical low-pass filter:

Formula:

- `filtered = alpha * raw  +  (1 - alpha) * previous_filtered`

The alpha parameter (0 to 1) controls the trade-off:

- alpha = 0.0 - completely frozen, ignores all new measurements
- alpha = 0.1 - very smooth, slow to respond to real changes
- alpha = 0.5 - balanced - good starting point for SO-ARM joints
- alpha = 1.0 - no filtering, raw signal passes straight through

Relationship to cutoff frequency:

- `alpha = 2*pi*fc*dt / (2*pi*fc*dt + 1)` `fc=cutoff freq, dt=sample interval`
- For SO-ARM at 50 Hz: fc=5 Hz gives alpha = 0.39

Start with alpha=0.2 and tune to your specific joint and motion requirements.

State Estimation

Kalman Filter

The Kalman filter is the gold standard for state estimation.

It runs two steps every cycle:

1 Predict

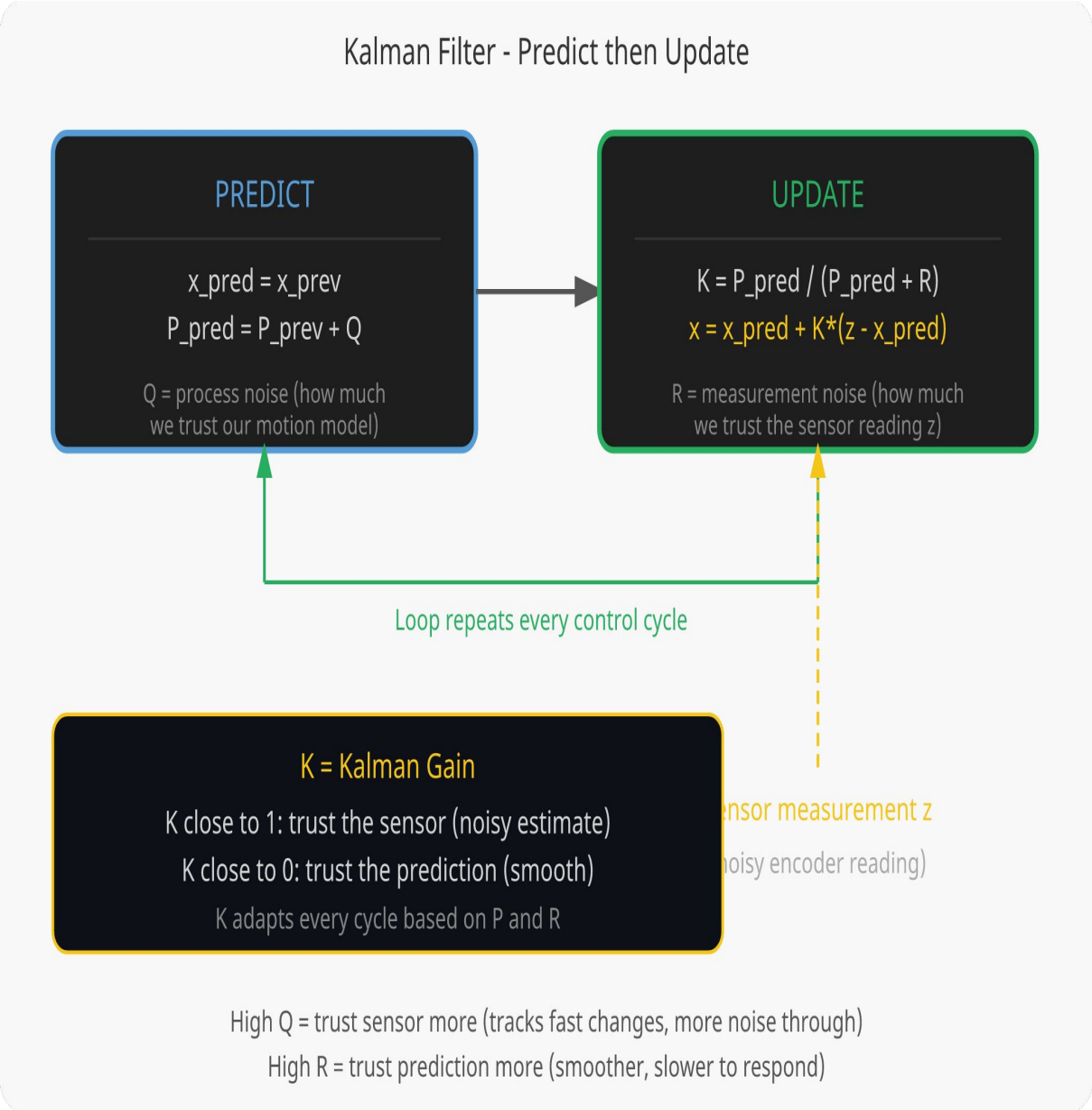
- Project forward using a motion model
- Uncertainty grows (P increases by Q)

2 Update

- Take the sensor measurement
- Blend prediction and measurement by Kalman gain K
- K is calculated automatically from P and R

Two parameters you tune:

- Q - how much you trust your motion model
- R - how noisy you believe the sensor is



Complementary Filter

Simpler than Kalman but works well for fusing two sensors:

Core idea:

- One sensor is accurate long-term but noisy short-term (encoder)
- Another sensor is accurate short-term but drifts long-term (gyro/IMU)
- Blend them using complementary weights that sum to 1

Formula:

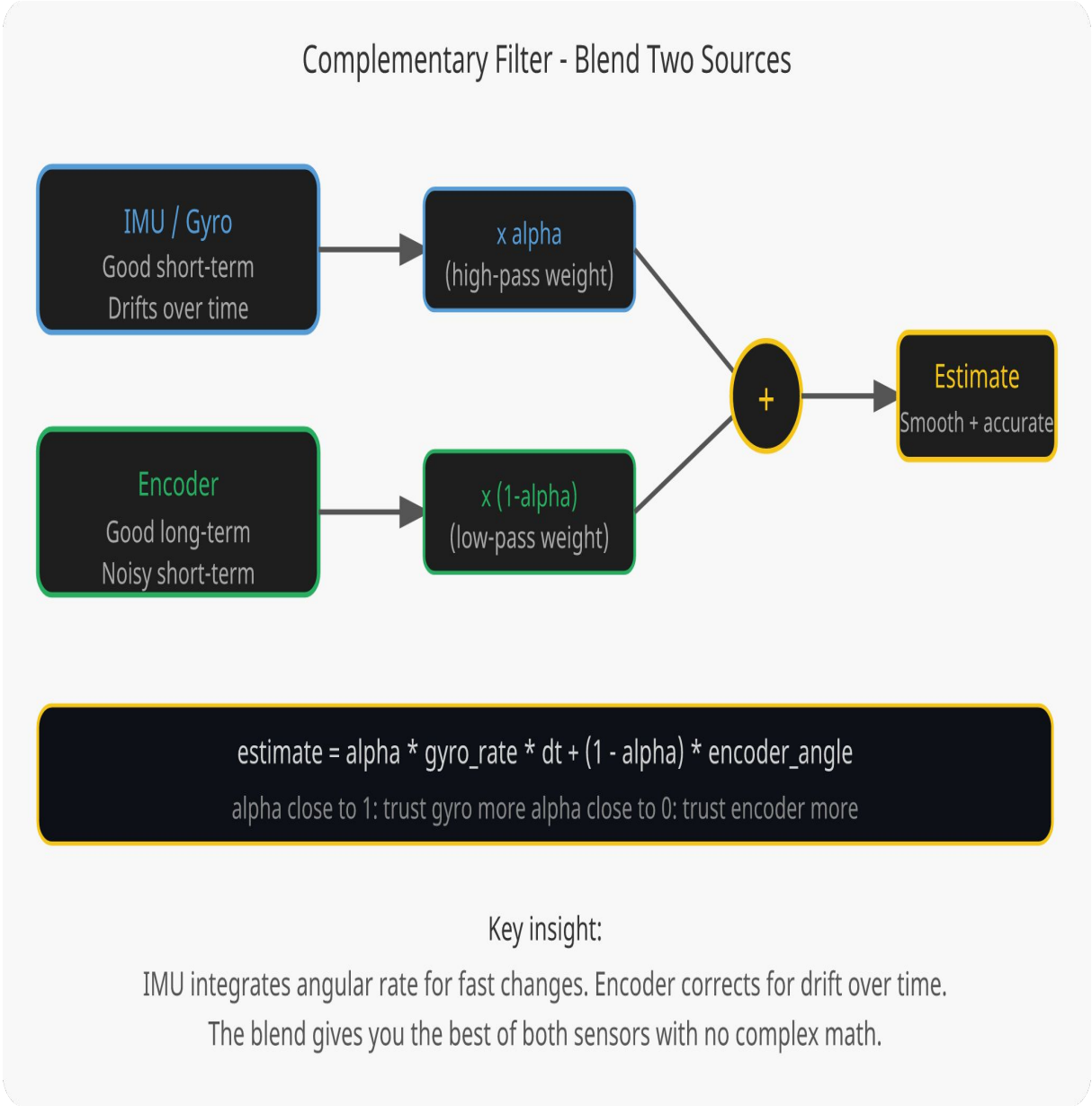
- estimate = alpha*(gyro*dt) + (1-alpha)*encoder

Alpha tuning:

- alpha=0.98: trust gyro heavily for fast motion, encoder for long-term correction
- alpha=0.5: equal blend, useful when both sensors have similar noise

Why use it over Kalman?

- Much simpler to implement and debug - one parameter vs two



Extended Kalman Filter - Brief Overview

The standard Kalman filter assumes everything is linear. Real robots are not.

Problem with linear Kalman on robots:

- FK and IK are non-linear (sin/cos functions) - linear Kalman gives wrong estimates
- Sensor noise distributions may not be Gaussian

Extended Kalman Filter (EKF) solution:

- Linearize the non-linear functions at each step using a Jacobian
- Run standard Kalman on the linearized version
- Re-linearize each time step as state changes

When is EKF used on the SO-ARM?

- Full 6-DoF state estimation combining encoder + IMU + camera
- When using velocity estimates derived from position differences (requires Jacobian)

For today: standard Kalman on a single joint position is sufficient and fully representative.

Error Propagation

Errors in measurements do not stay isolated - they combine and grow:

Single joint uncertainty

- If encoder has +/- 1 degree error, FK error for that joint is roughly +/- 1 degree at the tip

Chained joint uncertainty

- With 6 joints each with +/- 1 degree error, total end-effector error grows non-linearly
- Errors in base joints (shoulder) have larger effect than errors in wrist joints

Why filtering helps:

- Reduce each joint's position noise - every mm of improvement multiplies across the chain
- A filter that cuts noise by 50 percent on all joints can reduce end-effector error by much more

Rule: always filter at the earliest possible point in the pipeline.

Filtering after FK is too late - the error has already propagated through the math.

SO-ARM Application

SO-ARM Encoder Characteristics

Know your sensor before choosing a filter:

Feetech STS3215 encoder specs:

- 4096 ticks per revolution = 0.088 degrees per tick (fine resolution)
- Reports position as a 12-bit integer over the serial bus
- Read latency: roughly 1-2 ms per motor at 1 Mbit/s baud

Observed noise characteristics on the SO-ARM:

- At rest (no motion): typically +/- 1-3 ticks = roughly +/- 0.3 degrees RMS
- During motion: noise increases with speed due to motor EMI
- Wrist joints noisier than shoulder joints (smaller motors, less shielding)

Filter recommendation:

- Low-pass with alpha=0.2 to 0.4 for most joints
- Kalman with R=8.0, Q=0.05 as a starting point

Implementing a Low-Pass IIR Filter on SO-ARM Position

```
import time, csv
import numpy as np

# Low-pass filter parameters
alpha     = 0.2         # smoothing factor (0=frozen, 1=raw)
JOINTS    = robot.motor_names[:5]
dt        = 0.02        # 50 Hz loop

# Initialize filter state from first reading
raw_init  = robot.read('Present_Position', JOINTS)
filtered  = raw_init.copy().astype(float)

with open('filter_log.csv', 'w') as f:
    writer = csv.writer(f)
    writer.writerow(['time'] + [f'raw_{j}' for j in JOINTS] + [f'filt_{j}' for j in JOINTS])
    t0 = time.time()
    while time.time() - t0 < 10.0:
        raw      = robot.read('Present_Position', JOINTS).astype(float)
        filtered = alpha * raw + (1 - alpha) * filtered     # IIR step
        t        = round(time.time() - t0, 3)
        writer.writerow([t] + list(raw) + list(filtered))
        time.sleep(dt)
```

Implementing a Kalman Filter on SO-ARM Position

```
# Kalman filter - single joint version (shoulder_pan)
Q   = 0.05    # process noise: how much position can change per step
R   = 8.0     # measurement noise: encoder noise variance in deg^2

# Initialize
x   = robot.read('Present_Position', ['shoulder_pan'])[0]   # initial estimate
P   = 1.0     # initial uncertainty

def kalman_update(x, P, z):
    # Predict
    P_pred = P + Q
    # Compute Kalman gain
    K      = P_pred / (P_pred + R)
    # Update estimate
    x_new  = x + K * (z - x)              # blend prediction and measurement
    P_new  = (1 - K) * P_pred             # update uncertainty
    return x_new, P_new

while running:
    z    = robot.read('Present_Position', ['shoulder_pan'])[0]  # raw reading
    x, P = kalman_update(x, P, z)
    print(f'raw={z:.2f}  kalman={x:.2f}  gain={P/(P+R):.3f}')
```

Plotting Raw vs Filtered and Measuring Improvement

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv('filter_log.csv')

# Plot comparison for shoulder_pan
fig, axes = plt.subplots(2, 1, figsize=(12, 8), sharex=True)

axes[0].plot(df['time'], df['raw_shoulder_pan'],  label='Raw',      color='tomato',    alpha=0.7)
axes[0].plot(df['time'], df['filt_shoulder_pan'], label='Filtered', color='steelblue', lw=2)
axes[0].set_ylabel('Angle (deg)')  ;  axes[0].legend()  ;  axes[0].grid(True)

# Compute RMS noise (deviation from rolling mean)
window   = 20
raw_rms  = (df['raw_shoulder_pan']  - df['raw_shoulder_pan'].rolling(window).mean()).std()
filt_rms = (df['filt_shoulder_pan'] - df['filt_shoulder_pan'].rolling(window).mean()).std()
print(f'Raw RMS noise:      {raw_rms:.3f} deg')
print(f'Filtered RMS noise: {filt_rms:.3f} deg')
print(f'Noise reduction:    {(1 - filt_rms/raw_rms)*100:.1f}%')

plt.tight_layout()  ;  plt.savefig('filter_comparison.png')  ;  plt.show()
```

Raw vs Filtered - What You Should See

After running both filters and plotting:

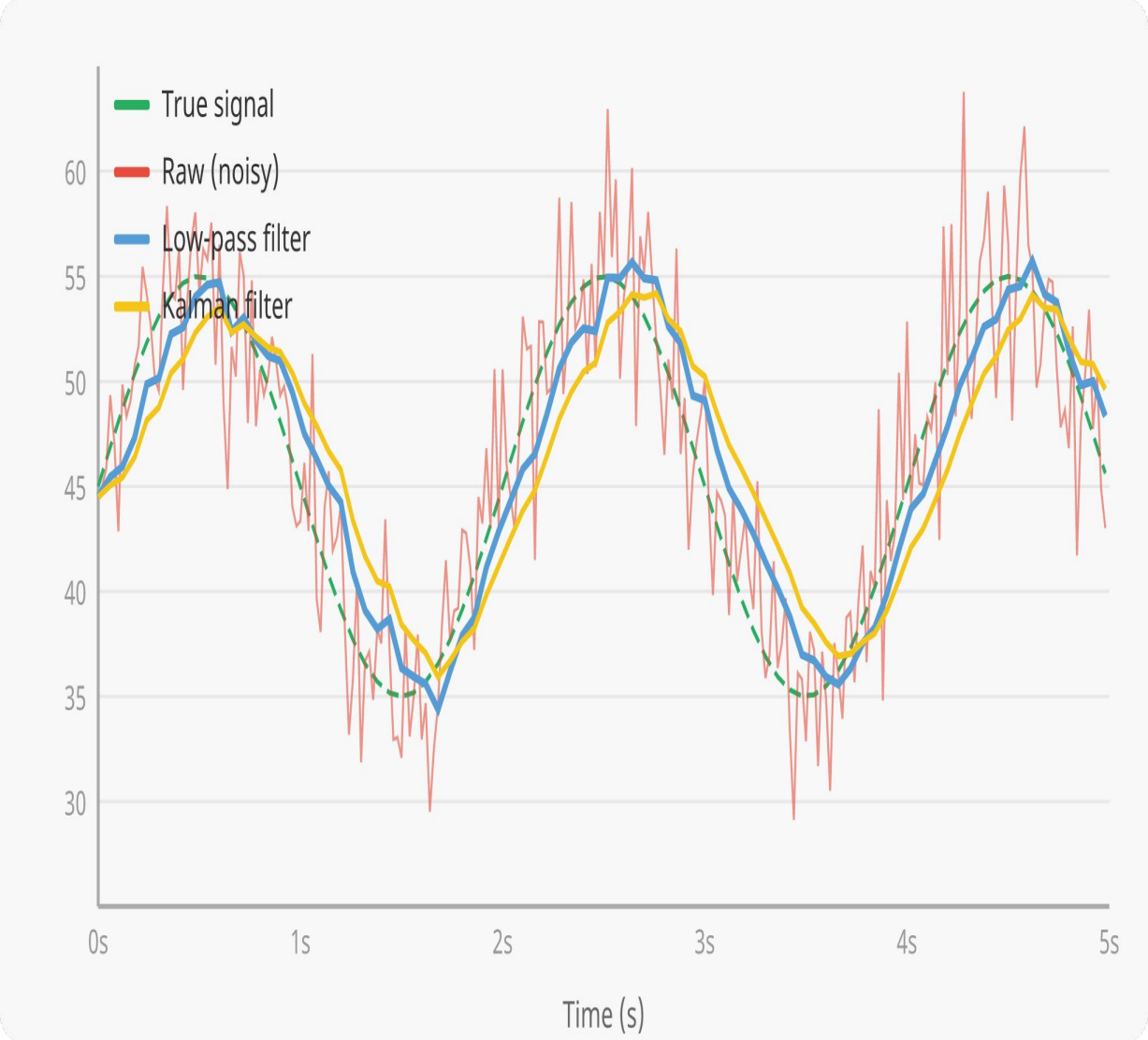
Green dashed - true underlying motion
Red - raw encoder (noisy, lots of spikes)
Blue - low-pass filtered (smooth, slight lag)
Yellow - Kalman filtered (smooth, tracks faster)

Key observations:

- Low-pass lags real motion slightly - alpha controls this trade-off
- Kalman adapts its gain dynamically - better during fast changes
- Both dramatically reduce RMS noise vs raw

When to pick which:

- Use low-pass for simplicity and real-time control
- Use Kalman when you need optimal noise vs. lag trade-off



Demonstration

Demonstration - Raw vs Filtered Sensor Data

Watch carefully - you will reproduce this in the hands-on.

What we will show:

- 1. Run the raw encoder loop at 50 Hz and plot live - observe noise at rest
- 2. Move the arm slowly by hand - observe motion signal buried under noise
- 3. Apply the low-pass filter with alpha=0.9 (too high) - noisy still
- 4. Apply alpha=0.2 - observe smooth signal tracking the real motion
- 5. Switch to Kalman filter - compare lag and noise between the two
- 6. Compute RMS noise for each and print the improvement percentage
- 7. Show what happens when you feed filtered position into the Week 6 PID - less motor chatter

Hands-On

Hands-On - Step 1: Implement and Log Both Filters

Goal: run the low-pass and Kalman filters simultaneously and log all signals.

1. **Set up the logging loop at 50 Hz with columns for time, raw, low_pass, and kalman**

2. **Initialize low-pass: filtered = first_reading.copy()**
- `alpha = 0.2   # start here`

3. **Initialize Kalman: x = first_reading, P = 1.0, Q = 0.05, R = 8.0**

4. **Run for 15 seconds while moving the shoulder_pan joint back and forth**
- Move slowly, then quickly, then hold still - cover all three cases

5. **Plot all three signals on the same chart**
- Include: raw (red, alpha=0.5), low-pass (blue), Kalman (yellow), on one chart

Hands-On - Step 2: Quantify and Tune

Goal: measure the noise reduction and tune alpha and Q/R until you hit the target.

1. **Compute RMS noise for each signal**
- Target: filtered RMS below 0.5 degrees on shoulder_pan at rest

2. **Tune the low-pass alpha**
- Try alpha=0.5 - does it meet the noise target? Try alpha=0.1 - does it lag too much?
- Find the highest alpha that still meets 0.5 degree RMS target

3. **Tune the Kalman Q and R**
- Increase R to 20 - smoother but slower tracking. Decrease to 2 - noisier but faster.
- Find Q and R that meet the noise target without noticeable lag during motion

4. **Connect to your Week 6 PID loop**
- Feed filtered position instead of raw - observe motor behavior improve
- Record Kp, Ki, Kd values that work well with filtered input

Common Mistakes

3 Errors You Will Definitely Hit

Every student hits these. Read this before starting.

Error 1 - Initializing filter state to zero

- If you start filtered=0 and the arm is at 45 degrees, the filter starts with a 45-degree error
- It takes many cycles to converge - arm appears to jump at start of logging
- `Always initialize: filtered = robot.read('Present_Position', JOINTS)[0]`

Error 2 - Wrong units in Kalman R parameter

- R must be in the same units as your measurement squared (degrees squared for angle)
- If encoder noise is roughly 1 degree, R should be around 1.0 to 4.0, not 0.01 or 1000
- Start with R = measured_noise_std^2 from your raw data

Error 3 - Applying filter after FK instead of before

- Filtering FK output in Cartesian space loses the per-joint noise model
- Always filter joint angles first, then run FK on the filtered angles
- The filtered angles feed cleaner data into every downstream algorithm

Deliverable

Deliverable - Generate Smooth Position Estimates from Noisy Data

Demonstrate all four checkpoints to a lab lead before leaving:

1 Both filters running and logging

- CSV has time, raw, low_pass, and kalman columns for shoulder_pan at 50 Hz

2 Comparison plot showing all three signals

- Raw, low-pass, and Kalman visible on one chart covering rest, slow motion, and fast motion

3 RMS noise target met

- Both filters achieve below 0.5 degree RMS at rest. Print the improvement percentage.

4 Filter parameters explained

- You can state your chosen alpha, Q, and R values and explain why you chose them.
- Bonus: show the PID controller behaving better with filtered input vs raw.